The for loop — repetition you can count

TC1037 · Algorithmic Thinking · Lecture 07

Dr. Leopoldo Rodríguez Salazar

Tuesday 13 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Los Alamos, 1946

Stanisław Ulam, recovering in bed, playing solitaire.

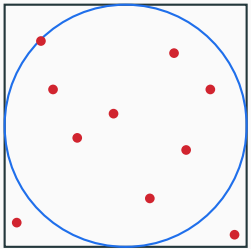
What is the probability that a game dealt at random can be won?

He tries to compute it. Gives up.

Why not deal a hundred games and count?

Algorithms to Live By, ch. 9 — *Randomness*

The napkin version



Square 2×2, circle of radius 1 inside.

Throw darts at random.

$$\frac{\text{inside}}{\text{thrown}} \approx \frac{\pi \cdot 1^2}{2 \cdot 2} = \frac{\pi}{4}$$

So $\pi \approx 4 \times \dfrac{\text{inside}}{\text{thrown}}$

Inside means $x^2 + y^2 \leq 1$. Nothing else.

Your turn — 3 minutes, on paper

The dart algorithm.

`INPUT:` how many darts `OUTPUT:` an estimate of π

You will need: a counter for the darts inside · a loop that runs *exactly* N times · one decision per dart.

Underline one line
The line that says **how many times** the loop runs.

That underlined line is today's keyword.

The same loop, twice

```
i = 1                    # start
while i <= 5:            # test
    print(i)
    i = i + 1            # step
```

```
for i in range(1, 6):
    print(i)
```

start, test and step — in one line you cannot forget.

6 is not included.

Three of last week's bugs cannot happen here. One new one can.

`range`, three shapes

`range(5)`	0 1 2 3 4	from 0, up to but **not including**
`range(2, 6)`	2 3 4 5	from, up to but **not including**
`range(10, 0, -2)`	10 8 6 4 2	from, up to, step
`range(3, 3)`	*nothing*	zero turns — not an error

`range(1, n + 1)` is "from 1 to n".

The `+ 1` is written every time, forever.

The accumulator, again — and its start value

```
def sum_to(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total
```

`sum_to(10)` $\rightarrow 55 \cdot$ `sum_to(0)` $\rightarrow 0$

```
def factorial(n):
    product = 1
    for i in range(1, n + 1):
        product = product * i
    return product
```

`factorial(5)` $\rightarrow 120 \cdot$ `factorial(0)` $\rightarrow$ **1**

The start value is *the answer when there is nothing to accumulate*: 0 for a sum, 1 for a product, 0 for a count.

A decision inside the repetition — count, keep, or skip

```
def count_multiples(n, limit):
    count = 0
    for i in range(1, limit +
        1):
        if i % n == 0:
            count = count + 1
    return count
```

`count_multiples(7, 100)` → 14

```
def sum_of_divisors(n):
    total = 0
    for d in range(1, n):
        if n % d == 0:
            total = total + d
    return total
```

`sum_of_divisors(12)` → 16

The `if` lives four spaces inside the `for`; the accumulator grows only on the turns that say yes.

Monte Carlo, in eleven lines

```
import random

def estimate_pi(trials):
    inside = 0
    for i in range(trials):
        x = random.random()
        y = random.random()
        if x * x + y * y <= 1:
            inside = inside + 1
    return 4 * inside / trials
```

`i` is never used inside — allowed: *do this `trials` times* is all we asked of it.

`random.random()`: a decimal from 0.0 to 1.0.

A flag

```
def is_prime(n):
    if n < 2:
        return False
    found_divisor = False
    for d in range(2, n):
        if n % d == 0:
            found_divisor = True
    return not found_divisor
```

Starts `False`.
Becomes `True` the moment the loop finds a reason to say no.
Never goes back.

After the loop, the flag *is* the answer, negated.

`is_prime(97)` True · `(9)` False · `(1)` False

No `break` in this course: a loop you leave from the middle is a loop whose exit you cannot read on the `for` line.

Three `for` loops that run — and are wrong

✗ 1 · the + 1

```
n = 5
for i in range(1, n
    ):
    print(i)
```

1 2 3 4. Where is 5?

✗ 2 · reset inside

```
total = 0
for i in range(1,
    4):
    total = 0
    total = total +
        i
print(total)
```

3, not 6.

✗ 3 · steering `i`

```
for i in range(3):
    print(i)
    i = i + 10
```

0 1 2. `for` hands out the next value regardless. Want to steer? That is a `while`.

In-class activity 3 — primes, counted · 20 min · graded

1. `is_prime(n)` — `True`/`False`; `n < 2` never prime. A `for` over `range(2, n)` and a **flag**. No `break`.
2. `count_primes(limit)` — how many primes from 2 to `limit` *inclusive*. A `for` that calls `is_prime`. Say why `limit + 1` appears.
3. `main()` — reads `limit`; below 2 → `Nothing to count.`; else the count.

`is_prime(1)`	False
`is_prime(2)`	True
`is_prime(9)`	False
`is_prime(97)`	True
`count_primes(10)`	4
`count_primes(2)`	1
`count_primes(1)`	0
`count_primes(100)`	25

Algorithm on paper first — both `FOR` ranges written, the flag named.
Program in exam format; shell tests under the algorithm.

Submit: photo + `A_Matricula_ic3.py` → Canvas → *In-class activity 3* — Sunday 18 Oct, 23:59.

Rubric: algorithm 40 · eight tests 40 · the `+ 1` and the `< 2` rule on purpose 20.

Early: why is `n // 2` enough? Why $\sqrt{n}$?

Before Friday

1. **Homework 5** — `is_perfect`, `longest_collatz` (a `for` around your HW4 function), `main`. One sentence per `for`: how many times it runs. **Sunday 18 Oct, 23:59.**
2. **Lab 4, Friday 16:30** — the academia's *for y while* lab: `factores_de`, `pares`, a menu, a `main` that repeats until *Salir*. One file `D_matricula.py`.
3. **Friday's session: `for` in practice — and the Situación problema is released.** Reread the brief; think about your partner.
4. **Stretch for credit** — handed in by Sunday? Grades go up this week. One use each.

Quick exam 4: Tuesday 20 October — the `for` loop.
Midterm: Friday 23 October — weeks 1 to 4.

for i in range(1, n + 1):

the + 1 is written every time, forever.